Dissertation

[TODO: create cover, add boilerplate stuff]

Acknowledgements

[TODO: write; may be added after uploading dissertation to MyPhD]

Table of contents

[TODO: fix which sections are included]

Samenvatting

[TODO: write; may be added after uploading dissertation to MyPhD]

Abbreviations and symbols

n	number of units or rows in a sample, indexed $i = 1, ..., n$
p	number of variables or columns in a sample, indexed $j = 1, ..., p$
Y	partially observed sample data (sized $n \times p$), also called the (hypothetically) **complete data**
R	response indicator matrix (sized $n \times p$), which stores the locations of the observed ($R = 1$) and missing ($R = 0$) parts of Y
Y_{obs}	observed data (elements of Y_{ij} where $R_{ij} = 1$), also called the **incomplete data**
Y_{mis}	**missing data** (elements of Y_{ij} where $R_{ij} = 0$)
Q	quantity of scientific interest; the scientific estimand we would like to estimate if we had complete data
$P(Q\|...)$	**complete-data model**; the analysis model we would like to fit if we had complete data, also called the 'model of scientific interest'
$P(R\|...)$	**missing data model**; the model that relates Y to R, which is interpreted as the 'probability to be missing'; also called the 'response model'
X	fully observed sample data for covariate(s) of Y
ψ	parameter(s) of the missing data model that represent any cause of the missingness unrelated to X and Y
MCAR	'Missing Completely At Random'; missing data mechanism with missing data model $P(R\|\psi)$, also called 'not data dependent'
MAR	'Missing At Random'; missing data mechanism with missing data model $P(R\|Y^{\text{obs}}, X, \psi)$, also called 'seen data dependent'
MNAR	'Missing Not At Random'; missing data mechanism with missing data model $P(R\|Y^{\text{obs}}, Y^{\text{mis}}, X, \psi)$, also called 'unseen data dependent'
LWD	list-wise deletion
CCA	complete case analysis
JM	joint modeling; imputation technique
FCS	fully conditional specification; imputation technique
SMC-FCS	substantive model compatible fully conditional specification; imputation technique
MICE	multivariate imputation by chained equations algorithm
`mice`	multivariate imputation by chained equations software
m	number of imputations, indexed $\ell = 1, ..., m$
$\dot{Y}_{\text{imp},\ell}$	imputed data in imputation number ℓ (of m total)
$\{Y_{\text{obs}}, \dot{Y}_{\text{imp},\ell}\}$	**completed data** in imputation number ℓ (of m total)

$\hat{Q}_\ell$	estimate of Q calculated from completed data in imputation number ℓ (of m total)
$\bar{Q}$	pooled estimate

Introduction

TL;DR (or: the quality of the solution)

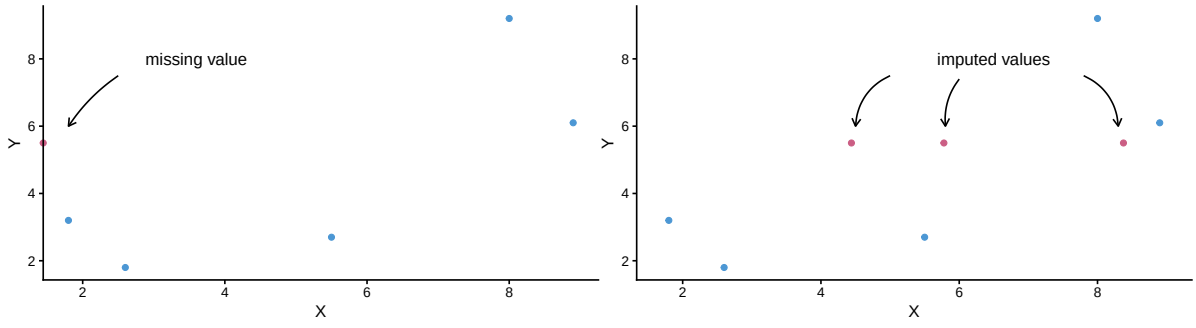
This dissertation is about algorithms and the quality of their output. I investigate how a specific type of data-generating algorithm can be evaluated for use in statistical inferences. The type algorithm in question are imputation algorithms. Imputation algorithms are algorithms with the purpose of generating plausible values based on a model of some observed information. This type of algorithm is popular for solving missing data problems.

Missing data problems are timeless, but technological advances have made treating missingness easier than ever. A single line of code can seem to solve your missing data problem nowadays. But is that solution really what you expect? That is what my dissertation is about. I want to contribute methodology that facilitates social and biomedical scientists in drawing valid inferences from incomplete data.

Imputation algorithms are designed to 'impute' (i.e., fill in) missing values in incomplete datasets. Incomplete data occur often in scientific disciplines with human subjects, such as the social science and biomedical research. A participant might, for example, drop out of a longitudinal study, or refuse to answer a sensitive question, resulting in gaps in the research data. These gaps, or missing values, complicate data analysis and may lead to biased and invalid study results. Missing data are ubiquitous and cannot be ignored without risking bias. Imputation is a popular and flexible procedure that separates the missing data problem from the analysis of scientific interest. Imputation allows data analysts to first solve for the missingness, to then be able to analyze the completed data as if they were completely observed.

The practice of drawing inference by replacing missing values with several synthetic substitutes is called 'multiple imputation'. Figure 1 provides an example of multiple imputation, where a missing value is replaced several times with plausible substitutes. The variability between imputations will reflect the uncertainty due to non-response in the final estimate.

Figure 1: Multiple imputation of a missing value



Although imputation is widely supported with software and many implementations of the method have convenient defaults, generating *good* imputations is not a trivial task. How can we know when to trust the output of imputation algorithms? The ideal route of studying the formal statistical properties of the method via analytical derivations is often impossible for imputation algorithms. Instead, the de facto standard way is to estimate their empirical performance via simulation studies, but these are not standardized across studies and therefore hard to compare. Moreover, simulation study results might not always translate to applied use. And for these real-world applications, there is no systematic format for evaluating the practical performance of imputation algorithms, such as diagnostics for algorithmic convergence. Ergo, evaluating whether imputations should be trusted remains challenging.

In this dissertation, I set out to investigate what is needed to develop, apply, and evaluate data-generating algorithms. I thereby focus on the imputation procedure 'MICE' as implemented in the R package `mice`.

Why? Because current evaluation practices (simulation design, convergence diagnostics, visual diagnostics) are fragmented and insufficient.

How? By bridging inferential evaluation and data-/algorithm-level diagnostics for imputation methods.

How did we get here? (or: statistical inference is just a tool)

My academic tag line has long been "statistician, interdisciplinarian, open scientist". And while there's no doubt I am an interdisciplinarian and an open scientist, I am no longer keen on calling myself a statistician necessarily. I am more inclined to say I am a methodologist rather than a statistician. As a methodologist, I am interested in how science is done, and how it can be done better. Especially in the social and biomedical sciences, research should not be purely quantitative. Statistics is just *one* way of doing science.

Don't get me wrong, statistics is still a favorite of mine. I love stats (I even have a mug that says so). Statistics is the science of uncertainty and extracting information from data (Hand, 2010). Extracting information from data and turn them into insights can feel magical. Statistical inference allows us to distinguish between real-world effects and random noise (Rosenberg & Curtain, 2015). I am most fan of the uncertainty of it all. I am motivated to contribute to trust in science by making it easier to reflect uncertainty in data analysis efforts, inferences, and graphs. In this dissertation, I hope to achieve this for missing data methodology.

Why are missing data problematic?

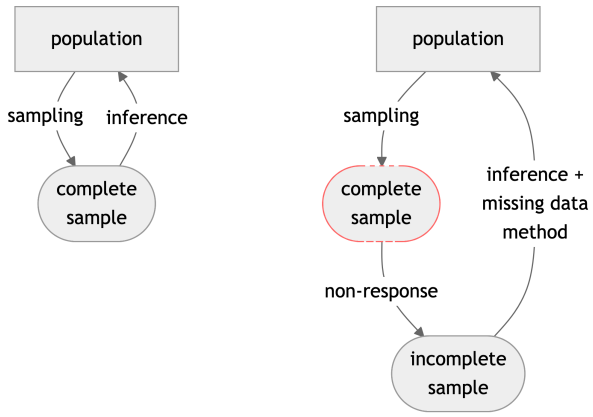
Statistical analyses are a means to infer conclusions about a population after observing only a subset of this population (a sample). If the sample is representative, sample estimates will reflect the population characteristics plus some uncertainty surrounding it. Statisticians call

this uncertainty the 'sampling error' due to observing only a subset of the population, which is often expressed as part of the 'standard error' (SE) surrounding the estimates. Standard errors then get translated into decision tools for inference such as p-values and confidence intervals (CIs), which help researchers distinguish between signal and noise. So, statistics is a means of estimating effects within a sample and inferring back to the population with some expression of uncertainty.

Generally speaking, statistical inference works really well. For example, when sociologists wanted to study trust in the government during the COVID-19 pandemic, it was not feasible to survey all Dutch citizens. Instead, they used a representative sample. Some thousands of people were asked to fill in a questionnaire about their trust in the government over the course of the pandemic. By repeating the questionnaire over time, the researchers wanted to gain insight into the development of (dis)trust among different subgroups in Dutch society. The estimates from this sample would have been used to infer back to the population and inform policy, if it had not been for missing data…

Missing data are a nuisance because it is impossible to do statistical inferences with incomplete data. When there is just one missing value in a dataset, statistical inference cannot be performed. With incomplete data, even simple statistics such as the mean of variables are mathematically undefined. To obtain results from your incomplete dataset, you will always employ a missing data strategy-–whether deliberately or not.

Figure 2: Inference with incomplete data



Note. Schematic view of statistical inference in the presence of non-response. Left: no missing data in the sample from the population (i.e., ideal circumstances/business as usual). Right: incomplete sample due to non-response, so no complete sample could be obtained. What types of missing data problems are we considering? We are interested in data with incomplete cases and/or variables, or 'item non-response'. Incomplete samples where some—but not all—entries are missing per row or per column. Entirely missing rows would be 'unit non-response'; entirely missing columns would be unobserved variables.

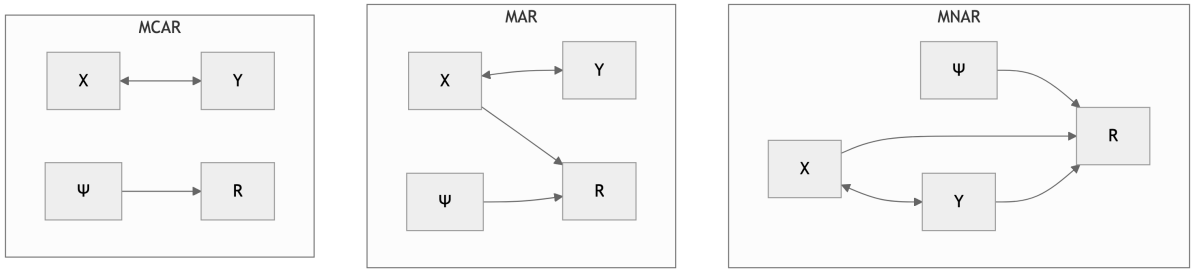
To get an estimate from incomplete data, some kind of missing data method has to be employed.

Even without specifying a missing data method explicitly, there will be handling of the missing data. The default strategy in most statistical software packages is to ignore all cases that have missing values (Van Buuren, 2018). This is referred to as 'list-wise deletion' (LWD). Ignoring missingness, however, lowers statistical power and may yield invalid inferences (e.g., biased estimates, spurious significant results; Van Buuren, 2018). List-wise deletion is ill-advised under most conditions (see van Buuren 2018 for exceptions). It is better to treat the missing data than to ignore it. List-wise deletion results in bias because it assumes that the observed part of the data is equivalent to the missing part of the data, and this is not usually the case.

Why are the missing and observed parts not equivalent? (or: missing data mechanisms)

[TODO: edit!] List-wise deletion (complete-case analysis) assumes that the observed part of the data is representative of the missing part of the data. In that study about trusting the government during the COVID-19 pandemic, it was not random who had missing data on the question about trust in the government. The technical term is 'Missing Not At Random'. The missingness depends on unobserved values themselves (i.e., people who distrust the government are less likely to respond to the trust item); this cannot be resolved from the observed data alone and requires assumptions or external information. If we can model the missingness probability based on known data, for example, previous observations or demographic data, we have 'Missing At Random'. That is, conditional on this information, we can get to 'missing at random' or 'seen data dependent'. This is in contrast to the 'unseen data dependent' missingness mechanism or 'missing not at random'. If there would be purely random missingness independent of individual characteristics, there is missing completely at random missingness mechanism, for example, due to a glitch in the service software. In human subjects research, pure MCAR is unlikely; methods that implicitly assume MCAR (like listwise deletion) are therefore likely to cause bias. In practice, it is impossible to know from the observed part of the data alone, so we have to make assumptions. So don't just use list-wise deletion. Treat the missingness to accommodate differences between the observed part of the data and the missing part.

Figure 3: Missing data mechanisms



Note. Y = incomplete data, X = fully observed covariate, R = missingness indicator for Y, ψ = causes of the missingness unrelated to X and Y (Schafer & Graham)

How to treat the missing data problem?

[TODO: edit!] There are three broad categories: deletion based methods (e.g. list-wise deletion), methods that incorporate the missingness into the model of scientific interest (i.e., the analysis model, such as a regression model), and imputation methods. Deletion-based methods do not generally accommodate differences between the observed and missing parts of the data, making them mostly applicable to MCAR. Modeling methods do accommodate MAR, but are often quite complex. For example, using 'Bayesian' methods that incorporate the missingness into the analysis model: they jointly specify the data model and missingness model per analysis. You have to specify the model for each analysis separately, and there are no null hypothesis significance tests which is a problem when working with applied researchers. Another option to incorporate the missingness in the analysis model is expectation maximization. But this yields many parameters and might not be available for a complex multi-level model like the Covid 19 data. Both of these techniques are tied to a specific analysis model, which might not be suitable for contemporary practitioners who run several models in a data science workflow. Imputation methods do not have these drawbacks of being computationally advanced and analysis-model specific.

Imputation separates the missing data problem from the analysis model and is the focus of this dissertation. Why impute? Imputation splits the missing data problem from the analysis of scientific interest: first solve missingness, then analyze completed data 'as if' they were fully observed. It allows the use of familiar complete-data methods instead of bespoke incomplete-data models. This helps when different people are responsible for different parts of a project (e.g., a missing data methodologist vs. domain expert). And when many different analyses are planned on the same data, it is possible to run several analyses without having to incorporate the missingness into the scientific model each time. This makes it better suitable within contemporary data science workflows than missing data methods that incorporate the missingness in the substantive model. Another advantage is that it is easier. Imputation is widely supported in software: one line of code can resolve the missing data problem at once. Compared to missing data methods that incorporate the missing data like Bayesian methods or EM-based approaches, imputation is more flexible and often less complex for non-experts. Compared to list-wise deletion, imputation methods offer the ability to accommodate MAR mechanisms, and they reduce research waste by retaining incomplete cases.

How does imputation work?

[TODO: edit!] We generate imputations to fill in the missing part of the data. We model the distribution of this missing part based on what is observed. So we use the observed part of the data to fix the missing part of the data. Which types of model or models do we use? The simplest model is no model at all. With hot deck imputation, we use random other person's observation to impute or fill in the values for people with a missing data point. The problem is that this disrupts the multivariate distribution of the data, so we get biased estimates.

Another simple technique is a univariate statistic such as the mean. Unfortunately, the mean will bias your scientific estimates even worse than list-wise deletion would, because there is no correction for the missing at random missingness mechanism. Mean imputation (and other univariate imputation methods) severely biases estimates and fails to account for MAR mechanisms.

Even 'advanced' techniques for substituting missing values that recover the unobserved value almost perfectly are problematic. Multivariate models such as stochastic regression imputation do correct for MAR missingness, but underestimate variance if used as single imputation. All techniques that rely on a single replacement value risk under-representing uncertainty in the inferences, because the uncertainty due to the non-response is not taken into account. We therefore have to repeat the stochastic draws to create a distribution of plausible values per missing datapoint, reflecting the uncertainty due to non-response.

Multiple imputation

MI is the practice of replacing each missing value with a synthetic substitute, multiple times. For each missing datapoint, we draw samples from a distribution of values that *might have* been (posterior predictive distribution). This generates several imputations: complete*d* datasets. The completed datasets can be analyzed as if the data were complete to begin with. On each completed dataset, we obtain estimates of the analysis of scientific interest. These estimates per imputation are then combined to obtain one pooled estimate. The variance of the estimate incorporates not just the sampling variance (within each imputation) but also added variance due to non-response (between imputation variance and simulation error). The variability between imputations reflects uncertainty due to the missingness.

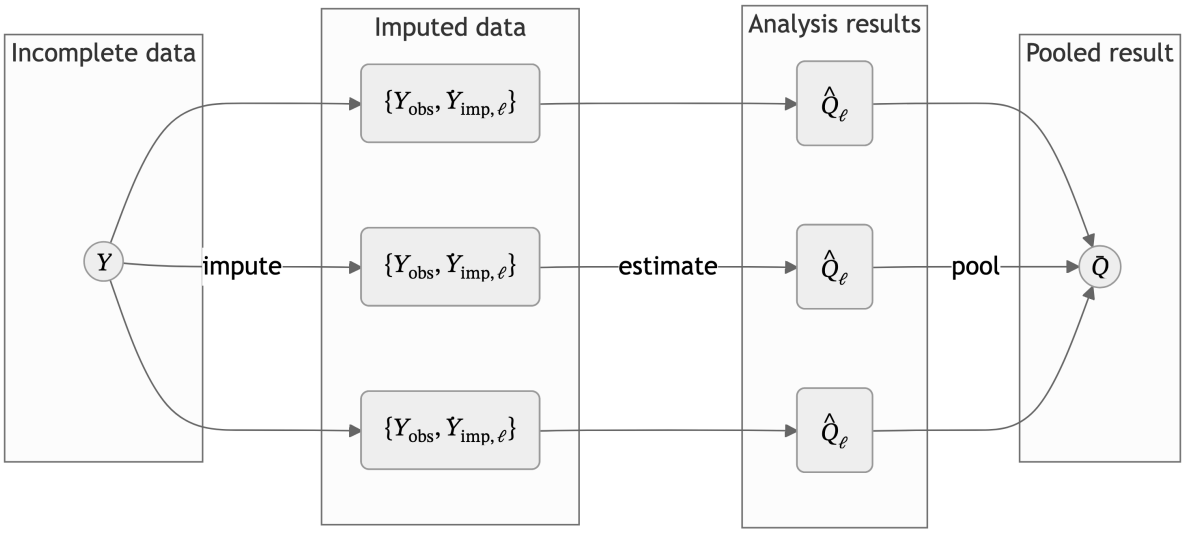
Multiple imputation means that each missing value is filled in several times, with slightly different model parameters to represent model uncertainty. The result of MI is that the uncertainty in the data due to missingness is preserved in the inference.

MI enables researchers with missing data problems to correct for differences due to missingness. Correcting for missingness happens at two levels: 1) modeling the relation between missing cases and covariates; 2) adding the appropriate variance to the models to represent the uncertainty due to missingness.

The result is a probability distribution for each missing value, from which we subsequently draw several values as imputations. If we would use infinitely many values as imputations, we would obtain the entire probability distribution. In practice, however, we use a limited number of imputations. With that, we approximate the probability distributions of missing values. The more imputations are used, the better we can represent the uncertainty in the data due to missingness.

Under the right conditions, MI yields valid, unbiased estimates (Schafer, 1997). The modeling assumptions inherent to a model-based technique like MI cannot be tested. Statistical models

Figure 4: Multiple imputation scheme ($m = 3$)



Note. Read left to right. Incomplete data Y is partially observed, $Y = \{Y_{\text{obs}}, Y_{\text{mis}}\}$. The missing data Y_{mis} is imputed m times to obtain m completed datasets $\{Y_{\text{obs}}, \dot{Y}_{\text{imp},\ell}\}$, where $\ell = 1, 2, \ldots, m$. On each imputation, the quantity of scientific interest Q is estimated, yielding m analysis results $\hat{Q}_\ell$. These results are pooled across imputations to obtain a single pooled result $\bar{Q}$.

like MI are optimized to distinguish between effect and noise of population averages–not the individuals that make up your sample.

FCS has no (deductive) proof, only by simulation.

What is FCS?

[TODO: reframe as 'Why MI in the flavor of mice?' I.e., FCS is a flexible and agnostic way to specify conditionals to arrive at a joint. MICE also allows JOMO: a direct and inflexible way to arrive at a joint. With MICE, it's easy to generate synthetic values. mice is not just imputation software, but rather an 'information generating machine'. Describe imputation models, analysis models, non-response model and SMC.]

How do we model the missing part of the data based on the observed part of the data? Historically, this would be done in two steps: a modeling step and an imputation step. In the modeling step, a multivariate distribution of the full dataset would be defined, after which imputations could be drawn from the joint model. The problem with joint modeling is that there might not exist a multivariate probability distribution, for example, with factor variables involved. But we do not need this joint model. Missing observations in incomplete cases/variables can be filled in using one or more imputation models instead. In the FCS

framework (Van Buuren et al., 2006), we only need an imputation model for each incomplete variable. FCS is substantive model agnostic, and does not assume any specific joint distribution. FCS will create an implicit joint model. While "Rubin (Ch. 5) distinguished three tasks for creating imputations under an explicit model: the *modeling* task the *imputation* task and the *estimation* task." (van Buuren, 2007, p. 221), FSC does not require these explicit tasks. Instead, FCS only requires an *imputation model* to describe how synthetic values may be generated, without explicit specification of the joint model. We define an imputation model per incomplete variable. And we need two things for this: 1) We need to specify the functional form of the model. For example a semi parametric method such as predictive mean matching for Likert scale items such as trust in the government. And 2) we need to choose imputation model predictors. For example, variables with a high linear association with this incomplete variable, or variables that are related to the missingness indicator of this incomplete variable (such as demographic data).

Imputation models consist of the functional form of the model (e.g., stochastic regression) and imputation model predictors (i.e., other variables in the data). Some things to keep in mind: With FCS, the quality of the imputation model can only be determined/assessed based on the complete data model. But what if that model is not (yet) available? Then take the most complex possible model and check that, because if you would take the simplest model, then that would implicitly mean you are constraining parameters (forcing b=0), systematically suppressing relations. Don't forget to add the effect of interest in your model of scientific interest. Otherwise, you are constraining the relation between the variables to be zero. If there is a specific data analysis model for the effect of scientific interest that you already have in mind, consider substantive model compatible fully conditional specification. SMC-FCS is optimized for known analysis model models: we assume a complete data model and derive the imputations from there. But what if the analysis model is unknown? Well, Fully conditional specification is the substantive model agnostic. So we build an imputation model based on the most complex analysis model that we might want to test.

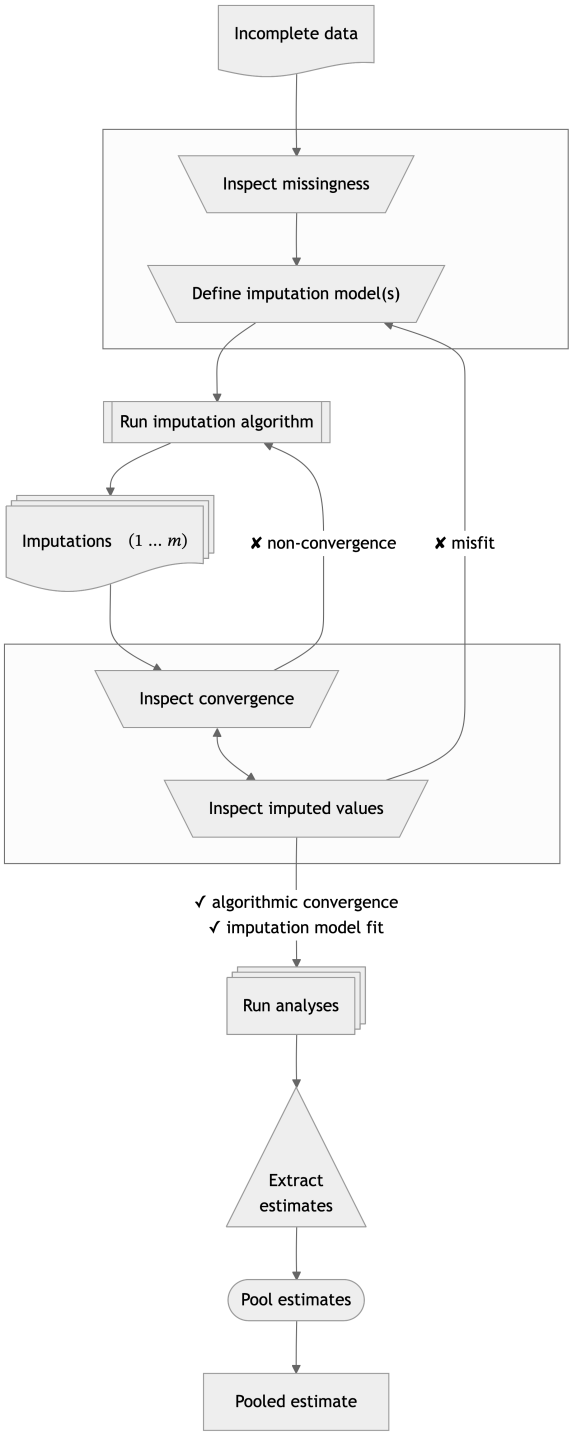
FCS implicitly relies on the joint distribution. The nice thing about joint modeling is that convergence of the model is guaranteed, whereas this is not the case with fully conditional specification. "Imputation models bypass the need to specify $P(Y, X, R)$, though their use creates new responsibilities for substantiating its correctness for a given statistical analysis." (van Buuren, 2007, p. 222). The question becomes: how do we choose an appropriate imputation model? And how do we know whether we have chosen an appropriate imputation model?

How to evaluate imputation models?

There are three main routes to evaluate imputation models. One can evaluate:

- the formal statistical properties of the method via analytical derivations;
- the empirical estimates of the method's attributes via simulation studies; and
- the practical performance in real data.

Figure 5: Multiple imputation flowchart



Note. Statistical evaluation is mainly focused on the relation between the incomplete data and pooled estimates. Computational evaluation is in every trapezoid: processes that require human intervention. In the first box, the question is: 'how can we model the missingness?', in the second box, the question is: 'do the imputation models fit the data?'. Missing in the graph: drawing inference with pooled estimate, and use external info after inspecting the missingness to involve all stakeholders (CRISP-DM steps).

The first and preferred option is to obtain the formal statistical properties of the imputation method using analytic derivations. Often this is impossible or unfeasible with fully conditional specification, when no known joint model exists. Without this, we cannot definitively conclude that an imputation method is an unbiased estimator. Instead, we estimate the empirical performance of these methods using simulation studies. This is the default evaluation in the missing data methodology field.

Simulation studies are the standard, but not standardized. If we have a specific analysis model of interest, we can calculate performance metrics such as bias and confidence interval coverage to conclude whether a method is unbiased and confidence valid for a certain estimand. This requires a specific analysis model, which might not be the case. And it is impossible to cover all the potential use cases. Another problem is that this is not standardized, so it's difficult to compare different simulation studies. We could also calculate the imputation accuracy by comparing the imputations against the complete data, but this is ill-advised.

The third option is the practical performance. This is case by case in applied use with real data applications. We check the imputation model itself, for example to see whether the algorithm has converged. We also inspect the output and we might run sensitivity analyses. This is not systematic at all currently. That is a problem. Users of the method then do not know whether they have correctly applied the method.

In this dissertation, I focus on the second and third routes. We will borrow some terms from related fields. I will connect it to concepts from systems and software engineering, and the information retrieval sub-field of computer science.

Computational evaluation, verification and validation

"Systems engineering (White et al. 1993; Stevens et al. 1998; Thayer 2002) is the activity of designing entire systems, taking into account the characteristics of hardware, software, and human elements of these systems. Systems engineering includes everything to do with procuring, specifying, developing, deploying, operating, and maintaining both technical and sociotechnical systems. Systems engineers have to consider the capabilities of hardware and software as well as the system's interactions with users and its environment. They must think about the system's services, the constraints under which the system must be built and operated, and the ways in which the system is used." (p. 552)

"system is more than simply the sum of its parts. … A useful working definition of these types of system is as follows: A system is a purposeful collection of interrelated components of different kinds that work together to deliver a set of services to the system owner and its users." (p. 556)

"The successful functioning of each system component depends on the functioning of other components" (p. 556)

–> 'interoperability' with other systems –> ggmice + ggplot2 + mice

“Human, social, and organizational issues affect the requirements and design of software systems”.

Systems engineering is dedicated to the study and development of systems “to support human activities” (Sommerville, 2015, p. 552).

“Systems that include software fall into two categories: 1. Technical computer-based systems [and] 2 Sociotechnical systems”. (Sommerville, 2015, p. 552) “For example, this book was created through a sociotechnical publishing system that includes various processes (creation, editing, layout, etc.) and technical systems (Microsoft Word and Excel, Adobe Illustrator, Indesign, etc.).” (p. 552)

“Sociotechnical systems: include one or more technical systems but, crucially, also people, who understand the purpose of the system, within the system itself. Sociotechnical systems have defined operational processes, and people (the operators) are inherent parts of the system.” (p. 552)

–> imputing is a sociotechnical system, not technical system, while mice is technical. (Sommerville, 2015, p. 552).

“sociotech systems “do not just include software and hardware but also people, processes, and organizational policies” p. 556)

“a sociotechnical system as a set of layers, where each layer contributes, in some way, to the functioning of the system. At the core is a software-intensive technical system and its operational processes” (p.557)

“Sociotechnical systems are complex systems, which means that it is practically impossible to have a complete understanding, in advance, of their behavior. This complexity leads to three important characteristics of sociotechnical systems: 1. They have emergent properties that are properties of the system as a whole, rather than associated with individual parts of the system. Emergent properties depend on both the system components and the relationships between them. Some of these relationships only come into existence when the system is integrated from its components, so the emergent properties can only be evaluated at that time. Security and dependability are examples of important emergent system properties. 2. They are nondeterministic, so that when presented with a specific input, they may not always produce the same output. The system’s behavior depends on the human operators, and people do not always react in the same way. Furthermore, use of the system may create new relationships between the system components and hence change its emergent behavior. 3. The system’s success criteria are subjective rather than objective. The extent to which the system supports organizational objectives does not just depend on the system itself. It also depends on the stability of these objectives, the relationships

(p. 558)

“Systems have properties that only become apparent when their components are integrated and operate together.”

In system life cycle research, there are standards for the evaluation of a system.

When the 'high road' of analytical derivations and proofs are impossible or unfeasible, computer scientists use computational evaluation to assess systems (e.g. algorithms). Computational evaluation encompasses a range of systematic, computer-based evaluation tools such as numerical experiments or simulation studies.

Verification asks 'was the system built right?', whereas validation asks 'was the right system built?'. So with verification, we can check the quality of a system by plugging in an input with a known output. Do we get the correct output in return? This is what we do with simulation studies and with unit tests in software. We want to check whether the system or the method in this case conforms to the requirements that we set (e.g. accuracy).

Then validation, on the other hand, asks whether the system is suitable for operation by the actual users in a specific intended use or application. Is a system able to accomplish its intended use? We can, for example, use integration tests.

How do methodologists know whether an imputation method works? Because we have empirical estimates from simulation studies (and, if available, formal properties from analytical derivations). How do imputers know whether their imputation models worked? Our current answer is well, they don't know for sure. And that's what this dissertation is all about.

verification and validation (V&V)

1. process of determining whether the requirements for a system or component are complete and correct, the products of each development phase fulfill the requirements or conditions imposed by the previous phase, and the final system or component complies with specified requirementscf. independent verification and validation

verification and validation (V&V) effort

1. work associated with performing the V&V processes, activities, and tasks [IEEE 1012-2012 IEEE Standard for System and Software Verification and Validation, 3.1.37]

validation test

1. test to determine whether an implemented system fulfils its specified requirements [ISO/IEC 2382:2015, Information technology — Vocabulary]

Building and evaluating imputation models

- Model building: how do we know which functional form to choose?
  - rely on defaults, tested in simulation studies
  - assess the distribution of the incomplete variable (e.g., visual inspection)
  - …? (maybe ad transformations?)

- Model building: how do we know which imputation model predictors to select?
  - association of incomplete variable with other variables in the data ("include an auxiliary variable if its correlation with the main variable is 0.5 (in absolute value) [25]" Nguyen et al., 2021)
  - association of missingness indicator with other variables in the data
  - external input (e.g., branching patterns, or expert knowledge to correct for MNAR)
  - Take the analysis model (if available) as starting point for imputation workflow, for congeneality
  - …?
- Model evaluation: how do we assess imputation model misfit?
  - fit of the model to the observed data
  - non-convergence in the algorithm
  - mismatch between observed and imputed data distributions (e.g., visual inspection)
  - …?

Evaluating imputations

- comp eval also possible without analysis model vs statistical evaluation relies on known analysis model (prangender vandaag want imputation gebruikt voor meer analyses) fcs is substantive model agnostic
- FCS has no analytical proof –> rely on evidence from simulation studies
- has been shown that it works under many conditions
- but how to know in practice? we need assumptions
- like assumptions for GLM, we look at the data to evaluate the appropriateness of our chosen model
- some assumption checks are visual inspection, some are quantitative, some need external info
- similarly, we need to evaluate the choice of imputation models
- ideally: evaluate inferential validity, but only available in simulation studies
- in practice, the thing we need is not available by definition
- some assumptions are impossible to definitively determine (e.g. MNAR), some we can evaluate by looking at the data
- this is what this dissertation is about: computational evaluation

- In practice, you cannot validate whether the imputation model was appropriate, because what you would need to do so is exactly the thing you don't have: the missing part of the incomplete data, while you only have the observed part.
- The best thing we can do is to rely on a combination of assumptions/theoretical justifications/substantive expertise and indirect evidence of imputation model fit: check the observed versus imputed data distributions, and inspect the imputations for signs of non-convergence. This dissertation offers insight and tools to do so.
- The only setting where we do have the comparative truth (the true but missing values) is simulation studies. Imputers have to rely on simulation study results in order to choose the best imputation method for their incomplete variables. That's why simulation studies for imputation methodology should be comparable with one another. Then, imputers can make a 'grounded' assessment which imputation methods may be applicable, and choose the most appropriate one. Example: NRI search for lower bound, so impute as 'positive behavior', but per definition wrong statistical inf, because we intentionally bias in one direction
- [in simulation study kun je waarheid maken, maar in de praktijk moet je aannames doen. die aannames zijn belangrijk, juist omdat ze niet/ten dele te bewijzen zijn]
- [what is a simulation study? add a brief methodological background section on simulation studies in statistics?]

What is computational evaluation?

- Computational evaluation is not equal to statistical evaluation. Statistical evaluation is important, e.g. to quantify bias and variance of estimators/procedures, but not the focus of this dissertation.
- Computational evaluation is something to check *conditional on* acceptable statistical properties, and does not replace statistical evaluation.
- Computational evaluation is originally from information retrieval research (see e.g. "Evaluation Measures (Information Retrieval)," 2025)
- The practice of computational evaluation combines *effectiveness* (i.e., did the process yield the correct result, e.g., accuracy, precision, recall, etc.), *system quality* (e.g., speed, coverage of all possible results, etc.) and, importantly, *user utility* (e.g., happiness, productivity, etc.) (Manning et al., 2008)
- The term 'computational evaluation' is not specifically defined for the field of statistics, but it could be useful.
- One definition of computational evaluation is "to determine the quality of solutions attainable" (Dudek, 2004)
- Translated to statistics, a model may be statistically valid, but not fit for purpose. For example, computations that take too long to converge for use in a production pipeline/real-time use in patient care.

- Another definition of computational evaluation is "the process of ensuring an algorithmic solution is a good one: that it is fit for purpose" (Curzon et al., 2014).
- We can use this when a statistical process has different purposes and thus different usability/requirements. For example, calculating uncertainty at the level of a single case, a sample, or inferring to a population.
- Since the term is not defined for statistics, there is no definition to use for the computational evaluation of imputation methodology.
- In this dissertation, I want to plant a flag: how can we define 'computational evaluation' in the context of imputation methodology?
- Working definition: *systematic assessment of the algorithmic behavior and data-level outputs of imputation procedures*, focused on: algorithmic stability (convergence, failure modes), sensitivity to modeling choices, plausibility and coherence of imputed values, and usability for applied researchers.
- These aspects are often already taken into account by imputers, but I try to systematize and operationalize these aspects that are often informal or *ad hoc*.

Dimensions of computational evaluation: e.g. inferential validity, algorithmic behavior, data-level plausibility, user utility, …

Why should we care about computational evaluation of imputation methodology?

- What do we care about? bias/coverage/variance, empirical relevance/plausibility, software-engineering nitpicks like speed/convergence/fault rate.
- Computational evaluation goes beyond statistical evaluation.
- In this dissertation I will use computational evaluation as a suite of tools to investigate the domains of algorithmic stability, data plausibility, and user burden, next to inferential validity.
- Even if we know the process yields correct inferences, we can evaluate the usability/appropriateness/plausibility of the solution. For example, one question to consider is: do the imputed values lie within the range of the observed values?
- Imputation theory does not require the imputed values to be plausible.
- Rubin (1976, 1987) developed the methodology to get correct population-level inferences from incomplete samples, not at the individual case-level within the sample. [TODO: consider adding that since then, data analysis has changed]
- Bluntly stated: imputation pragmatists/purists don't care for the imputed values themselves, as long as they result in statistically valid inferences.
- But what are the limitations of purely inferential metrics (bias, coverage)? e.g. imputing negative values for age, using PMM for a variable with a sparse region (yielding 'unrealistic' imputed values), or evaluating imputed values of non-converged chain (?), or not knowing the analysis model up front so congeniality is at risk (???) -> PPC would show problem with sparse region

- And vice versa: when might the plausibility of imputed values mislead a data analyst into believing the imputations are statistically valid? when 'looking reasonable' is confused with 'being consistent with the data-generating process', such as single imputation (stoch reg), or imputing under MCAR assumption when there is a MAR mechanism, or destroying the joint distribution of the data while preserving the marginals. [TODO: figure out the difference with Gerko's interpretation of joint occurrence (cross tabs) within conditional distribution; should we make an algorithm to show user which combinations of bivariate obs only occur in imp? Not now!]
- Statistical evaluation of bias and coverage is not possible in practice, only in simulation studies.
- Stat evalualtion may not possible IRL? Bias and confidence interval coverage are impossible to determine without a comparative truth (although PPC and bootstrapping allows us to compare with respect to observed values).
- [TODO: rephrase as ability to explain differences between observed and imputed data (e.g., lower means in boys data), not as reverse engineering but as statistical evaluation of the data –> then computational eval may be needed to explain differences?] Computational evaluation means not just taking the answer as-is, but tracing back the solution of an algorithm to its origin/back-tracing the procedure: you should *always* be able to retrieve the chain of the imputation procedure. E.g., implausible or impossible imputed values can yield valid statistical inference, but are we satisfied with the way that the imputation were obtained?

Examples of stat but not comp

- polynomials (ignoring the U in imps could still get stat val)
- example: psych study with baseline, some of which are missing; baseline, intervention, follow-up –> mice would inflate baseline and follow-up if both are incomplete; statistical evaluation will not show this (unbiased, cov ok, etc.); over-imputation or postpred check would show diff distr for obs and imp; circularity should be removed: follow-up should not predict baseline
- fcs iterations not stable, but estimand ok (in simulation study true value) –> conv at cell level??
- PMM with part of sample space with too little donors

What is the scope of this dissertation?

RQ: what tools are required to assess the quality of data-generating algorithms? in the development, evaluation, and application.

- is the status quo sufficient? no, statistical evaluation is required, but not sufficient.
- how Rubin developed the methodology works in theory, but is not enough for contemporary practitioners

- can we use existing tools or do we need more? and how? no, not all are present in the imputer's toolbox. I contribute to the new status quo ("verleggen" = generalizable scientific contribution of this dissertation)
- unifying how imputations are generated and evaluated (??)
- lifecycle of imputation methods: method is theorized, method is implemented in software, method is evaluated in simulation study, method is put to scrutiny by scientific community, method is applied in practice, method is evaluated in real-world data.
- two types of evaluation: statistical properties of the method in simulation (e.g., bias and confidence-validity; statistical evaluation) & practical application (computational evaluation).

[TODO: remove what I will not address! Maybe add a sentence 'You might expect me to go into…' In the category non-response: just item, and M(C)AR mech. In the category imputation algo: just FCS. In the category substantive models: just GLM for stat inference.] In this dissertation, we will focus on missing data problems with item non-response and MCAR/MAR missingness mechanisms. Unit non-response and MNAR missingness mechanisms are outside of the scope of this dissertation. The missing data method under consideration is primarily multiple imputation by chained equations (MICE), but some results may generalize to other iterative (FCS) imputation algorithms. We do not cover joint modeling (JOMO) imputation or non-imputation missing data methods. Although MICE is not SMC-FCS, we will restrict the analysis of scientific interest mostly to GLM family of models. The applications are aimed at statistical inference (not prediction or causal inference).

Chapters

1. Evaluation paper: evaluating (new) imputation methods via simulation studies/how we know FCS works
2. Convergence paper: evaluating imputation algorithm quantitatively
3. Software: choosing/evaluating imputation models in practice

Introducing chapter 1: evaluation

- **RQ: what to look out for when designing simulation studies for the evaluation of imputation methodology?**/How can computational evaluation criteria be systematically incorporated into simulation studies for imputation methodology?
- published as Oberman & Vink (2024), cited 27x according to Google Scholar (of which 11x CrossRef)
- based on literature review Liu et al. (2023)
- How can we trust simulation studies for imputation methods?
- How can we make them compatible & fair?

REF: Morris ADEMP: Using simulation studies to evaluate statistical methods, pre-registration template, FIMD, Introduction to statistical simulations in health research?, Pitfalls and Potentials in Simulation Studies?

Introducing chapter 2: convergence

- **RQ: FSC is iterative, so convergence *should* be a requirement for valid inference, or is it?**/How does non-convergence in iterative imputation affect inferential validity, and how can non-convergence be diagnosed?
- simulation study, published as pre-print (11x cited according to Google Scholar), presented at ICML
- even without convergence, we can get valid inference at sample/population level, but not at individual cell level
- imputation stays a custom job: convergence of imputation is conditional on imputation model
- today, imputation are not generated for 1 analysis prob, but all analyses that may be fitted on the data –> all imputation models should be compatible and congenieal –> more 'omvattende' imputation models required
- liefst wil je analysemodel parameters tracken, daarom is computational evaluation zo belangrijk
- Can we quantify the convergence of the MICE algorithm?

REFs: Graphical and numerical diagnostic tools to assess suitability of multiple imputations and imputation models, Diagnostic checking of multiple imputation models, Convergence Properties of a Sequential Regression Multiple Imputation Algorithm, FIMD, Diagnostics for multivariate imputations

Introducing chapter 3: `ggmice`

[TODO: add poster and 'age old adage' picture vs 1000w.]

This chapter introduces the software `ggmice`: an R package for building and evaluating imputation models. The `ggmice` package offers tools for the visualization of incomplete data, imputation models, and imputed data.

Indicators for adoption: how is the software already used?

- software published on Zenodo (Oberman, 2022), with open code and bug reporting via GitHub
- peer-review not formal via journal, but via Issues and Pull Requests on GitHub (10x external Issue, 13x external PR)
- impact: citations (5x according to Google Scholar), CRAN downloads (852x/month; 21k total) and conda downloads (100+/month; https://anaconda.org/channels/conda-forge/packages/r-ggmice/overview)

Design principles

- `ggmice` is compatible with the popular R packages `mice` (for imputation) and `ggplot2` (for flexible data visualizations based on the 'grammar of graphics' framework).
- `mice` imputation models are substantive model agnostic: flexible, var-by-var definition of imputation model. `ggmice` supports this.
- Grammar of Graphics = layered plots (Wilkinson, 2012) –> `ggmice` produces `ggplot` objects: layered, easily adjustable
- Assumptions: imputation model should fit the observed part, imputation model should be congenial –> providing tools to see observed and imputation side-by-side/evaluation of the distribution of observed and imputed data

Limitations

- make explicit what these plots/analyses can and cannot tell us: imputation models should fit the observed part of the data, but no way to determinately state miss mech.
- visualization =/= objective truth –> good imputation model fit/plausible values is not a guarantee for statistical validity
- visualization is not a replacement of evaluation but support, also check metrics such as FMI
- interpretation remains subjective (e.g. convergence, MAR assumption, etc.)

What does ggmice aim to solve?

The `ggmice` package aims to solve the following problems:

- no direct graphical comparisons between incomplete and imputed data; side-by-side presentation of the data before and after imputation; Visualizations with `ggmice` can be made to inspect marginal and joint distributions of incomplete variables side-to-side to compare incomplete and imputed data

- no previously existing methods for visualizing `mice` imputation models; complex imputation models are easier to review through visualization (as opposed to console prints/excel exports of predictor matrix)
- existing plotting tools for `mids` objects not easily editable/publication-ready; Visualizations with `ggmice` can be extended and adapted with tidyverse functions to create publication-ready graphs of imputed data (reflecting the uncertainty due to missingness).
- unified toolbox for the development and evaluation of imputation models

Existing solutions

The `ggmice` package fills a hiatus in the software landscape, with other packages generally supporting either `mice` or `ggplot2`, but not both. Table REF provides an overview of R packages that offer visualization tools for incomplete and imputed data. In short: with `mice` you cannot plot incomplete data distributions, all other packages lack support for `mice`-imputed data, and visualization of `mice` imputation models did not exist at all prior to `ggmice`.

	ggmice	mice	naniar	VIM
Incomplete data (e.g., `data.frame` object)				
– joint & marginal distributions	+	−	+	+
– missing data patterns	+	+	±	+
– imputation models	+	−	−	−
Imputed data (e.g., `mids` object)				
– joint & marginal distributions	+	+	−	−
– imputation models	+	−	−	−
– trace plot of the imputation algorithm	+	+	−	−
User utility (e.g., `ggplot` object)				
– flexibility/extensibility of visualization types	+	−	±	−
– adaptability/publication readiness of visualizations	+	−	+	−

TODO

- write table footnote to explain all aspects of the table. i.e., these are only diffs. the table does not include things that all packages can –> maybe as appendix?
- explain the purpose of the table: I'm both player and referee. I've got skin in the game, but I made the rubric. Someone else might evaluate/value different aspects.
- also add number of downloads: `ggmice` has 800 monthly compared to `VIM` 13k and `naniar` 22k
- make 'Imputation models' a separate section?

Software development

- Open Source Software development/FAIR –> R, GitHub, CRAN, Zenodo, …
- CRAN downloads
- GitHub stars and issues
- StackOverflow mentions/questions
- feedback at conferences
- citations of the package (Zenodo reference)

Future research & development

- visualizing `mira` objects
- add posterior predictive check plots & over-imputation
- quantifying uncertainty at pattern/cell level
- tools for sensitivity analyses: inducing MNAR in observed part of the data and evaluating effects on the imputations/inference
- What data-level diagnostics are informative for assessing imputation model adequacy in practice (i.e., when inferential validity cannot be evaluated)?
- pre vs post imputation facilitator functies om te checken of er nieuwe verbanden in zitten (kruistabel)

Chapter 1: evaluation

[TODO: link/insert]

Chapter 2: convergence

[TODO: link/insert]

Chapter 3: ggmice

[TODO: link/insert]

Discussion

Conclusion

- Computational evaluation of imputation methodology is never finished.
- The main findings in this thesis are…
  - Defining and testing new imputation methods through simulation studies requires careful consideration of the simulation design and evaluation to make imputation methods comparable across studies. We need simulation studies to know imputation methods work. But we cannot be certain they actually *did* work whenever we apply them. The missing part of the data is by definition not available, so we cannot definitively state anything about the imputation model fit to the unobserved part of the data. Moreover, there's untestable assumptions (e.g., MNAR) and lack of clear diagnostic cut offs (e.g., convergence).
  - Non-convergence is hard to diagnose, and typical thresholds to evaluate non-convergence may not be applicable to FCS algorithms: MICE reaches stable output before non-convergence metrics would indicate so. There is a mismatch between theoretical convergence criteria and practical algorithmic behavior. That's why we still have to rely on visual inspection.
  - Visual inspection aids imputers in developing and evaluating imputation models, because formal tests are not available. The R package `ggmice` offers tools for the computational evaluation of the imputations models that were previously unavailable.
- Recommendations for fellow missing data methodologists:
  - design simulation studies structured and reproducible (e.g., report design choices, publish code)
  - talk to your intended audience (i.e., applied researchers), or at least check what they are struggling with (e.g., StackOverflow)
  - …?
- Recommendations for fellow imputers:
  - think before you code
  - look at the data, plot the data
  - …?

Limitations

- Computational evaluation is not a substitute for thinking. It may give a false sense of certainty, e.g. against MNAR mechanism (untestable assumption).

- Instead, use computational evaluation as complement to (not a replacement for) substantive knowledge and sensitivity analyses. Use expert insight
- This entire dissertation relies on the R language and centers the `mice` package [TODO rephrase as not a flaw: I focus on a dominant ecosystem, and concepts generalize beyond].
  - While this set-up is very popular, the data science landscape has expanded and is ever evolving
  - Many machine learning and deep learning methods are not (yet) implemented as imputation models in `mice`. These might be able to better approximate the posterior predictive distribution of the missing values than `mice` methods, but research should show that still.
- ~~I haven't tested whether the tools I developed actually improve the imputations of `mice` users~~ This dissertation evaluates tools and diagnostics at the methodological level; assessing their causal impact on user behavior and imputation quality is not feasible to systematically test.
  - GitHub issues and StackOverflow are indication
  - Onmogelijk om evidence-based conclusies te trekken, maar misschien wel evidence-informed?
- The visualizations implemented in `ggmice` are generally better suited for numeric data, as opposed to categorical variables (or even open text field data, which is not supported by `mice` currently). Future research on: associations of cat vars with miss indicators, and convergence parameter other than SD of imputed values.

> - TODO: rephrase 'haven't's into future research (positive phrasing: reframe as scope conditions, not shortcomings)
> - maybe future: computational evaluation of variable importance within imputation models

Outlook

- This chapter ends with a future perspective…
- Missing data remains a problem indefinitely. I expect imputation methodology to be developed and applied for the foreseeable future. But the way we deal with data might change.
- With the rise of machine learning and deep learning methods (or "AI"), evaluation will become ever more important. Novel methods are being developed under a prediction framework, not statistical inference framework: ignoring the uncertainty in the estimates,

and thus not incorporating between-imputation variance. This leads to too narrow CIs, and too low p-values. Which, in turn, causes spurious results.

- AI models should propagate uncertainty!
- trust in science requires honest reflection of uncertainty in our analyses
- be open about uncertainty, and limitations of scientific studies
- publish null results (e.g., registered reports or pre-registrations)
- share data and code with other scientists, and the public
- publish open access, educational materials too!
- change the way we evaluate science: this dissertation is an example of the new recognition and rewards vision. it includes software as an actual chapter, not something extra.
- I hope to inspire others to reflect on their own view of what scientific output is. And most of all, I hope to set an example for other PhD candidates to do the same
- imputation is not always the way to go: missingness can tell you about mismatch between data collection method and lived experience –> positionality??

References

Curzon, P., Dorling, M., Ng, T., Selby, C., & Woollard, J. (2014). *Developing computational thinking in the classroom: A framework*. Computing at School.

Dudek, G. (2004). Computational Evaluation. In G. Dudek (Ed.), *Collaborative Planning in Supply Chains: A Negotiation-Based Approach* (pp. 165–213). Springer. https://doi.org/10.1007/978-3-662-05443-7_7

Evaluation measures (information retrieval). (2025). In *Wikipedia*. https://en.wikipedia.org/w/index.php?title=Evaluation_measures_(information_retrieval)&oldid=1329650960

Liu, D., Oberman, H. I., Muñoz, J., Hoogland, J., & Debray, T. P. A. (2023). Quality Control, Data Cleaning, Imputation. In F. W. Asselbergs, S. Denaxas, D. L. Oberski, & J. H. Moore (Eds.), *Clinical Applications of Artificial Intelligence in Real-World Data* (pp. 7–36). Springer International Publishing. https://doi.org/10.1007/978-3-031-36678-9_2

Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press. https://doi.org/10.1017/CBO9780511809071

Nguyen, C. D., Carlin, J. B., & Lee, K. J. (2021). Practical strategies for handling breakdown of multiple imputation procedures. *Emerging Themes in Epidemiology*, *18*, 5. https://doi.org/10.1186/s12982-021-00095-3

Oberman, H. I. (2022). *Ggmice: Visualizations for 'mice' with 'Ggplot2'* [Computer software]. Zenodo. https://doi.org/10.5281/zenodo.6532702

Oberman, H. I., & Vink, G. (2024). Toward a standardized evaluation of imputation methodology. *Biometrical Journal*, *66*(1), 2200107. https://doi.org/10.1002/bimj.202200107

Rubin, D. B. (1976). Inference and Missing Data. *Biometrika*, *63*(3), 581–592. https://doi.org/10.2307/2335739

Rubin, D. B. (1987). *Multiple Imputation for nonresponse in surveys*. Wiley.

van Buuren, S. (2007). Multiple imputation of discrete and continuous data by fully conditional specification. *Statistical Methods in Medical Research*, *16*(3), 219–242. https://doi.org/10.1177/0962280206074463

Van Buuren, S., Brand, J. P. L., Groothuis-Oudshoorn, C. G. M., & Rubin, D. B. (2006). Fully conditional specification in multivariate imputation. *Journal of Statistical Computation and Simulation*, *76*(12), 1049–1064. https://doi.org/10.1080/10629360600810434

Wilkinson, L. (2012). The Grammar of Graphics. In *Handbook of Computational Statistics* (pp. 375–414). Springer, Berlin, Heidelberg. https://doi.org/10.1007/978-3-642-21551-3_13

CV

[TODO: write]